Geavanceerde Computerarchitectuur: Project Object Trajectory

Arno De Lathauwer en Indra Coppens

May 2025

1 Context

In dit project kozen we ervoor om de baan van een object in een video te volgen. We begonnen met een basisimplementatie in Python en C++, waarna we overstapten naar CUDA om de bewerkingen op de GPU uit te voeren. Tot slot pasten we meerdere technieken toe en vergeleken we deze met elkaar.

2 Probleemstelling

Het probleem dat we aanpakken, is het volgen van de baan van een object in een video. Als input krijgen we een videofragment waarin een rond object beweegt. We kozen ervoor om deze inputvideo zelf te genereren, zodat er duidelijke kleurcontrasten zijn tussen het object en de achtergrond. In een realistische toepassing zou dit echter een opgenomen video kunnen zijn, bijvoorbeeld van een biljartspel, een bowlingbal of een tennismatch. Het doel is om als output een lijn te tekenen die de afgelegde baan van het object weergeeft. Deze lijn wordt geprojecteerd op het eerste frame van de video, om de visualisatie overzichtelijker en duidelijker te maken.

Deze toepassing werd gerealiseerd door de RGB-waarden van de pixels in elk videoframe te analyseren. Pixels waarvan de kleur binnen een vooraf gedefinieerd bereik valt, specifiek opgesteld voor het te volgen object, worden beschouwd als een 'hit'. Nadat alle rijen en kolommen zijn geïdentificeerd waarin dergelijke pixels voorkomen, wordt het middelpunt van deze regio bepaald. Het midden wordt bepaald door het midden te nemen van alle kolommen waar deze kleur in voorkomt, en analoog het midden te nemen van alle rijen waarin deze kleur voorkomt. Dit middelpunt wordt vervolgens opgeslagen als de positie van het object in dat specifieke frame. Deze procedure wordt herhaald voor alle frames in de video, met als doel het volledige traject van het object te reconstrueren. Tot slot worden alle verzamelde coördinaten gevisualiseerd om het uiteindelijke traject van het object duidelijk weer te geven. Een mogelijke output wordt weergegeven in *Figuur 1.*

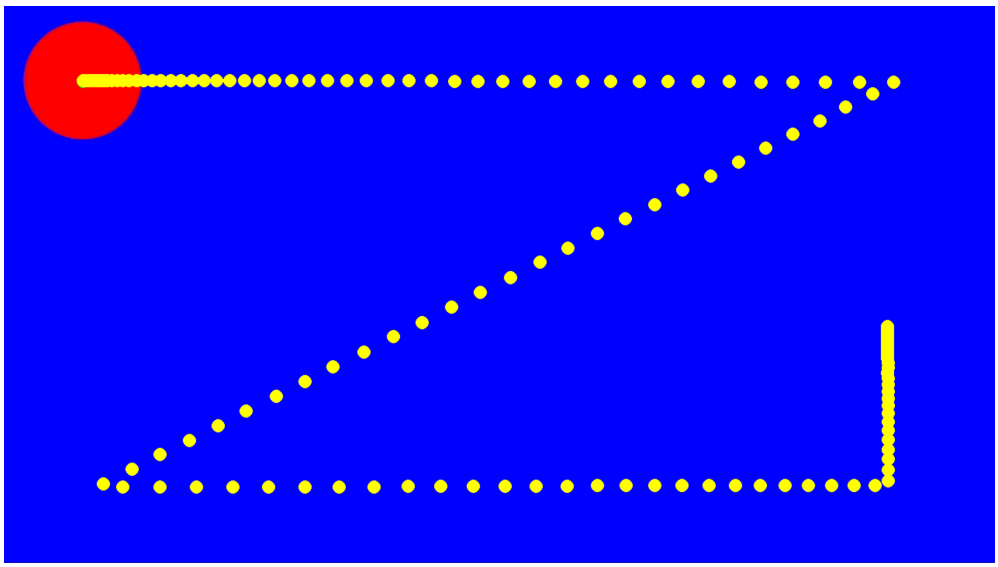


Figure 1: Output

3 Waarom dit probleem met de GPU aanpakken

Dit probleem kan in praktische toepassingen onderhevig zijn aan strikte tijdsbeperkingen. Een voorbeeld hiervan is het gebruik als hulpmiddel voor scheidsrechters in bepaalde sporten, waarbij real-time verwerking vereist is. Om die reden leek dit probleem ons bijzonder geschikt om te implementeren op de GPU, met als doel zo snel mogelijke resultaten te bekomen.

Daarnaast biedt dit probleem talrijke mogelijkheden voor parallelle verwerking. Zo kunnen bijvoorbeeld alle pixels afzonderlijk worden geëvalueerd om te bepalen of hun kleur binnen het vooraf bepaalde bereik valt. Ook kunnen meerdere opeenvolgende frames simultaan worden verwerkt in parallelle batches. Bovendien laten ook de afzonderlijke R-, G- en B-kleurkanalen zich onafhankelijk van elkaar vergelijken, wat bijkomende parallellisatie mogelijk maakt.

4 Methodologie

4.1 Implementatie in Python en C++

In eerste instantie werd een implementatie ontwikkeld in zowel Python (Appendix A) als C++ (Appendix B), met als doel de algemene werking van het algoritme beter te begrijpen en later tijdsvergelijkingen te kunnen uitvoeren. Aangezien de Python- en C++-versies sterk analoog zijn in opbouw en functionaliteit, beperken we ons in deze bespreking tot de C++-implementatie. Dezelfde werking is eveneens terug te vinden in de Python-versie.

Als eerste stap wordt de video omgezet naar een reeks afzonderlijke frames. Deze C++-code is dan geschreven om de positie te bepalen van een gekleurde bal in deze reeks PPM-afbeeldingen, waarbij gebruik wordt gemaakt van eenvoudige kleurfiltering op basis van RGB-waarden. Aan het begin van het programma worden enkele constanten gedefinieerd, waaronder de resolutie van de ingelezen beelden en het bereik waarbinnen elk van de kleurwaarden zich moet bevinden om tot het object (bal) te behoren.

Elk beeld wordt afzonderlijk ingelezen via de functie *get_image_array_ppm*, die een PPM-bestand opent, de headerinformatie uitleest en de ruwe pixeldata in een dynamische array van *uint8_t* opslaat.

Vervolgens wordt voor elk frame een tweedelige analyse uitgevoerd om te bepalen waar de bal zich bevindt. Twee vectoren, respectievelijk voor rijen en kolommen, worden gebruikt om het aantal pixels dat voldoet aan de kleurvoorwaarden per rij en per kolom bij te houden. Per pixel wordt gecontroleerd of de rode component binnen het opgegeven bereik ligt, en indien dat het geval is, wordt vervolgens ook gecontroleerd of de groene en blauwe componenten binnen hun respectievelijke grenzen vallen. Indien aan alle voorwaarden voldaan is, wordt zowel de bijbehorende rij als kolom geteld als mogelijke locatie van de bal. In de vectoren (voor rij en kolom) komen er dan éénen op de locatie waar het object is gedetecteerd en nullen waar dit niet het geval is (zie *Figuur 2*).

Na deze detectie worden de gemiddelde rij en kolom berekend op basis van de locaties waarin geldige pixels zijn gedetecteerd. Dit gebeurt door het gemiddelde te nemen van alle rijen en kolommen waar ten minste één overeenkomstige pixel is gevonden.

$$x_{\text{midden}} = \frac{x_1 + x_2}{2} \quad \text{en} \quad y_{\text{midden}} = \frac{y_1 + y_2}{2}$$

De gemiddelde waarden worden beschouwd als de centrale coördinaten van de bal in het huidige frame. Deze coördinaten worden opgeslagen in een vector van paren, die uiteindelijk na afloop van alle frames wordt weggeschreven naar een tekstbestand via de functie *save_coordinates_to_file*. Een Python-programma plaatst deze punten dan op het eerste frame van de video.

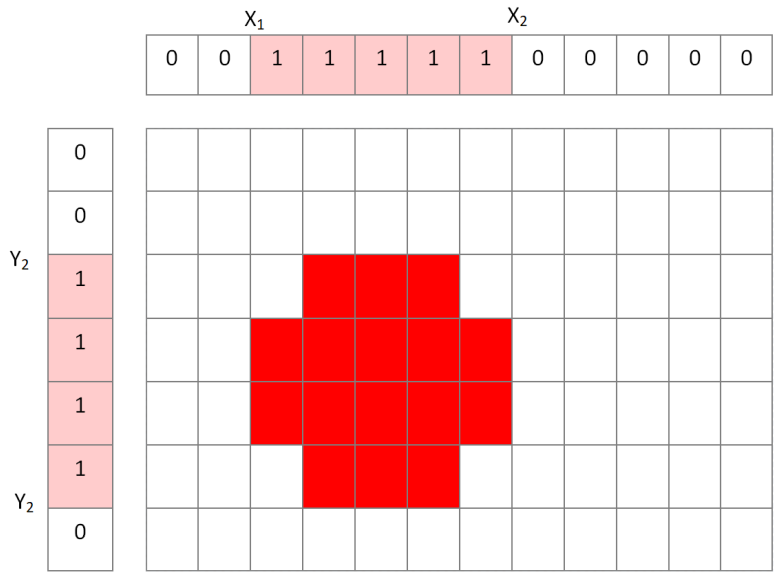


Figure 2: Vector per rij en kolom voor gedetecteerde pixels

4.2 Overzicht van de geïmplementeerde versies

In *Tabel 1* is een overzicht van de geïmplementeerde versies te zien. In het vervolg van dit verslag en in figuren spreken we regelmatig over versie x, dit verwijst naar deze nummers.

Versie	**Beschrijving**
Versie 0	Eerste implementatie in CUDA: 1 thread per rij en 1 thread per kolom
Versie 1	Toevoegen van dubbele reductie
Versie 2	Efficiënt VS. inefficiënt gebruik van warps
Versie 3	Elke pixel (RGB) krijgt een aparte thread
Versie 4	Elke kleurcomponent krijgt een eigen thread
Versie 5	Optimalisatie op versie 3
Versie 6	Asynchrone operatie

Table 1: Overzicht van de geïmplementeerde versies

4.3 Eerste implementatie in CUDA

Deze CUDA-versie heeft een gelijke input en output als het C++ programma, maar hier wordt het bepalen of een pixel binnen de kleurenrange valt verschoven naar de GPU. Het gemiddelde van de gedetecteerde kolommen en rijen bepalen wordt hier nog op de CPU uitgevoerd.

In het begin van het programma worden opnieuw enkele constante parameters gedefinieerd, waaronder de beeldresolutie (1280 x 720), het aantal kleurkanalen (3 voor RGB) en de onder- en bovengrenzen van de kleurwaarden die overeenkomen met de rode bal. De structuur van het programma blijft grotendeels gelijk: 150 opeenvolgende frames (horend bij de video) worden ingelezen, verwerkt en de coördinaten van de bal worden opgeslagen.

Het belangrijkste verschil zit in de functie *find_object_colour*, die als CUDA-kernel wordt uitgevoerd op de GPU. Deze kernel maakt gebruik van een eenvoudig threadmodel waarbij de eerste H threads verantwoordelijk zijn voor het tellen van kleur-overeenkomende pixels per rij, terwijl de volgende W threads hetzelfde doen voor de kolommen. Elke thread krijgt dus een unieke index i, en afhankelijk van de waarde van i beslist de kernel of een rij of kolom moet worden geanalyseerd (`i < H = rijen, H <= i < W+H = kolommen`). Met andere woorden worden alle rijen en kolommen in parallel doorlopen en zal elke pixel twee maal bekeken worden. Dit is geïllustreerd in *Figuur 3*. In dit voorbeeld zijn de eerste 7 (hoogte, H) threads de threads voor de rijen en de volgende 12 (breedte, W) de threads voor de kolommen. Omdat elke thread slechts één waarde in zijn respectievelijke row_counts of col_counts array aanpast, is er geen nood aan atomic operaties, wat de kernel eenvoudiger en sneller maakt. Deze implementatie is te vinden in Appendix C.

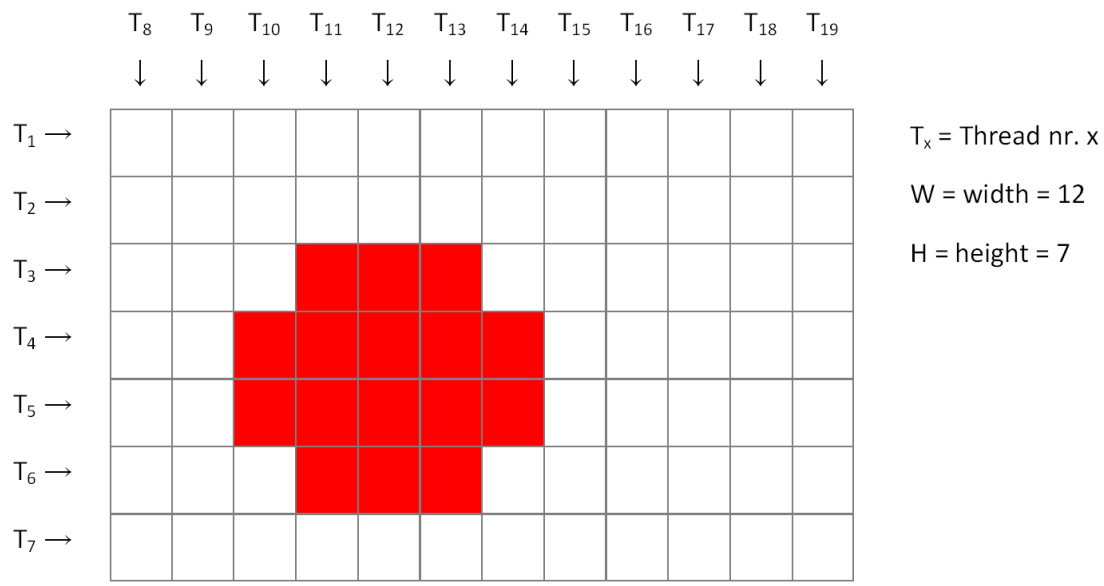


Figure 3: Verdeling threads (CUDA versie 1)

Het main-programma coördineert de overdracht van data tussen CPU en GPU. Per frame wordt het beeld ingelezen en naar de GPU gestuurd via cudaMemcpy. Daarna worden geheugenblokken gealloceerd voor zowel de pixeldata als de teller-arrays. Na het uitvoeren van de kernel (*`find_object_colour<<<...>>>`*) worden de resultaten teruggekopieerd naar de host, en wordt het bepalen van de gemiddelde rijen- en kolomnummers uitgevoerd op de CPU. Deze stap gebeurt hier nog niet op de GPU, wat we in verdere versies wel nog zullen doen. Na afloop van alle frames worden de gedetecteerde coördinaten opgeslagen in een uitvoerbestand.

4.4 Dubbele reductie

In de oorspronkelijke implementatie (versie 0) werd het middelpunt van een object in een afbeelding bepaald op de CPU. Dit gebeurde door de laagste en hoogste index te identificeren waarop pixels binnen de gewenste kleurrange aanwezig waren, en vervolgens het gemiddelde van deze indices te berekenen.

In de nieuwe implementatie (versie 1) wordt deze berekening verplaatst naar de GPU. Het doel is om de minimale en maximale index van de gedetecteerde pixels te bepalen via parallelle reductie-operaties. Door vervolgens het gemiddelde van deze twee waarden op de CPU te berekenen, verkrijgen we het middelpunt. Omdat de minimale en maximale reductie gelijktijdig worden uitgevoerd, noemen we deze aanpak dubbele reductie. Deze bewerking gebeurt afzonderlijk voor zowel de kolom- als rij-index arrays. De dubbele reductie wordt uitgevoerd door de kernel `min_max_index`. Daarnaast kan de verkorte code gevonden worden in Appendix D.

De reductie begint vanaf een array waarin enkel de indexen worden opgenomen van kolommen (of rijen) waar pixels werden gedetecteerd binnen de juiste kleurrange. Deze array werd opgevuld via de `find_object_colour`-kernel die ook werd gebruikt in versie 0 om de kleuren te detecteren. Via een parallelle reductieoperatie worden de minimum- en maximumwaarden naar respectievelijk het begin en einde van de array geschoven. De uitleg kan gevolgd worden op *Figuur 4* met bijhorende legende voor de threads op *Figuur 5*.

De kernel wordt uitgevoerd met één block met `N/2` threads (waarbij `N` het aantal elementen in de array is). Er worden $\log_2$(`size`) iteraties uitgevoerd (waarbij `size` gelijk is aan het aantal elementen in de array). Voor een array met 16 elementen hebben we dus vier iteraties nodig, zie *Figuur 4*.

Tijdens de eerste iteratie verwerkt elke thread twee waarden en schrijft het minimum naar links en het maximum naar rechts. In ons geval zouden we deze stap kunnen overslaan omdat het hier om geordende indices gaat. Echter, omdat de kernel mogelijk hergebruikt wordt voor andere doeleinden, is een volledige min-max-reductie geïmplementeerd.

Een belangrijk detail is dat als een van de twee waarden nul is, de niet-nul waarde naar beide posities geschreven wordt. Dit garandeert dat nul niet als minimum wordt beschouwd, maar de laagste niet-nul waarde. Een voorbeeld hiervan is zichtbaar bij thread 4 in *Figuur 4*. Deze aanpak betekent echter ook dat als het object zich op rij- of kolomindex 0 bevindt, dit niet correct wordt gedetecteerd en hier de laagste index 1 zal zijn. Dit probleem

kan verholpen worden door de array vooraf te initialiseren met een waarde zoals -1, maar in deze context is een dergelijke nauwkeurigheid niet noodzakelijk.

Vanaf de tweede iteratie zijn de operaties voor minimale en maximale reductie volledig gescheiden. In elke volgende laag zijn dubbel zoveel threads vereist vergeleken met een enkele min- of max-reductie. In *Figuur 4* duiden de rode threads de minimale reductie aan, die de kleinste waarde naar links schrijven. De groene threads voeren analoog de maximale reductie uit en schrijven de grootste waarde naar rechts.

Bij de minimale reductie worden de eerste `pow(2, log_2_size)/(pow(2,iteratie))`-threads gebruikt, met volgende indices in de array:

- Index element 1: `int val1 = idx * pow (2 , iteratie ) ;`
- Index element 2: `int val2 = idx*pow(2,iteratie) + (pow(2,(iteratie-1)) ;`

Voor de maximale reductie worden de volgende `pow(2, log_2_size)/(pow(2,iteratie))`-threads gebruikt, met volgende indices in de array:

- Index element 1: `int val1 = N - 1 - (new_idx*pow(2,iteratie) + (pow(2,(iteratie-1)))) ;`
- Index element 2: `nt val2 = N - 1 - new_idx*pow(2,iteratie);`

Hierbij is idx de threadindex, iteratie het iteratienummer en N het totale aantal elementen in de array.

De beschreven GPU-kernels maken gebruik van __syncthreads(), wat enkel werkt voor threads binnen hetzelfde blok. Wanneer men meerdere blokken gebruikt, kunnen er synchronisatieproblemen ontstaan. In deze toepassing vormt dit echter geen probleem, aangezien de breedte van de verwerkte frames 1280 pixels bedraagt, wat overeenkomt met 640 threads — een aantal dat ruim binnen één blok past.

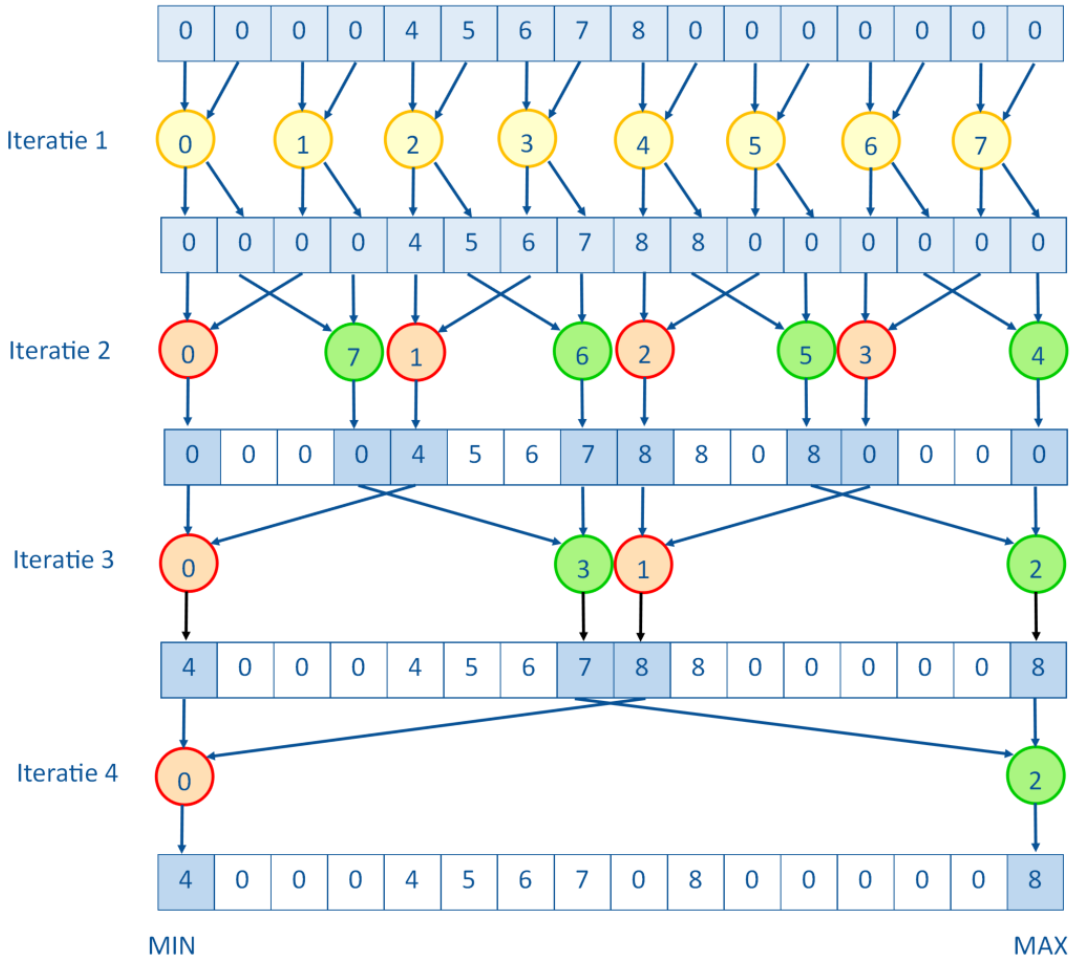


Figure 4: Dubbele reductie

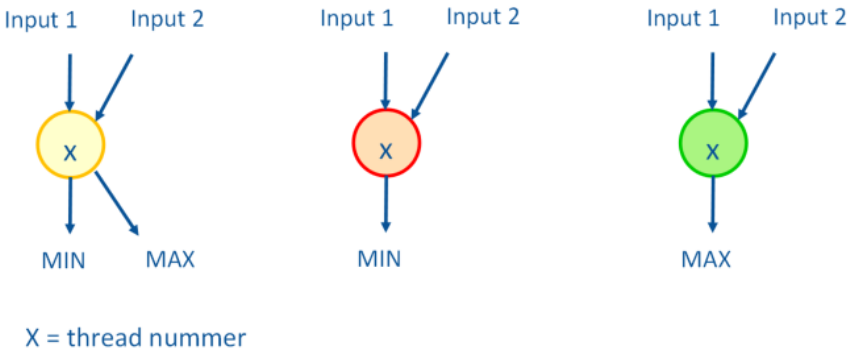


Figure 5: Legende threads dubbele reductie

4.5 Warp efficiency

Deze versie is volledig analoog aan versie 0 op vlak van functionaliteit, maar er werd specifiek aandacht besteed aan hoe de kernel opgeroepen werd. De eerste keer werd de kernel opgeroepen met 2 blocks van elk 1024 threads. Dit levert 32 warps per block op (64 warps in totaal) waarbij alle 32 threads in een warp aan het uitvoeren zijn. Vervolgens werd de kernel opgeroepen met 2000 blocks en telkens één thread per block. Dit levert 1 warp per block op (2000 warps in totaal) waarbij telkens maar één thread per warp effectief aan het uitvoeren is. Alle andere threads in deze warp zijn aan het idlen. De relevante delen van de bijhorende code kunnen gevonden worden in Appendix E.

4.6 "Overclocking" the GPU

In deze versie worden meer threads aangeroepen dan dat er effectief gelijktijdig kunnen worden uitgevoerd op de GPU. Als we ervan uitgaan dat we werken met een GT 740, deze heeft twee stream multiprocessoren met elk 192 cores, kunnen we twee blokken van elk 1024 threads in parallel uitvoeren. Dit komt neer op 2048 threads die effectief in parallel uitgevoerd kunnen worden. Wanneer we meer threads aanroepen dan dat er effectief in parallel kunnen worden uitgevoerd, worden blocks sequentieel uitgewisseld en krijgen ze elk een bepaalde uitvoertijd op de stream multiprocessoren van de GPU.

In versie 3 wordt er voor iedere pixel een thread voorzien, dit resulteert in 921600 (720 * 1280) threads voor het analyseren van één frame. Iedere thread controleert dan telkens de drie kleurencomponenten van iedere pixel en zal op basis hiervan (indien de pixel binnen de vereiste kleurrange valt) de waarde op de overeenkomstige rij en kolom index ophogen in de twee vectoren die het aantal pixels dat voldoet per rij en per kolom bijhouden (zie C++ implementatie). Aangezien we nu meerdere threads hebben (er zijn meerdere pixels op een rij/kolom) kunnen meerdere threads de waarde op eenzelfde index in deze vectoren aanpassen waardoor we de waarde op deze index moeten ophogen via een `atomicAdd()`. Dit introduceert enige overhead maar uit eerdere labosessies weten we dat deze overhead relatief gezien klein is. Een andere mogelijkheid zou zijn om aan iedere thread opnieuw een unieke index te geven (we hebben dan een entry per pixel) in beide vectoren (door de vectoren groter te maken) en deze dan achteraf te postprocessen. De snelheidswinst die we hiermee kunnen maken is echter miniem in vergelijking met het werken met atomic operaties. Om de complexiteit te beperken en de leesbaarheid te verhogen hebben we gekozen om te werken met atomic operaties. De relevante delen van versie 3 kunnen gevonden worden in Appendix F.

In een nog extremere variant (versie 4) wordt aan iedere kleurcomponent een thread toegewezen die evalueert of de kleurcomponent binnen de correcte range (voor deze kleurcomponent) valt. Hierdoor wordt de kernel opgeroepen met maar liefst 2764800 (720 * 1280 * 3) threads. In dit scenario moeten de threads die de R-, G- en B-component van eenzelfde pixel verwerken van elkaar weten wat het resultaat was van de kleurcomponent-evaluatie. Dit werd bereikt door in shared memory voor ieder block een array `component_flags` met size 1023 te voorzien.

De grootte van 1023 werd hier specifiek gekozen omdat dit deelbaar is door drie. Door de kernel vervolgens op te roepen met 1023 threads per block zorgen we ervoor dat alle threads binnen eenzelfde block een volledige pixel verwerken. Per block worden er 341 pixels verwerkt, dit moet een geheel getal zijn omdat anders de kleurcomponent flags van de 342ste pixel verdeeld zitten over twee blocken en shared memory is enkel gedeeld over één block. Hierdoor zouden we voor pixels waarvan de kleurcomponenten verwerkt worden door threads uit een verschillend block niet kunnen bepalen of deze pixel binnen de kleurrange valt.

Eens alle kleurcomponenten gecontroleerd zijn, wordt er via de thread die de rood waarde evalueerde gecon-

troleerd of de evaluatie van de volgende twee threads, die de groen en blauw waarden evalueerden, ook true hebben opgeleverd in de `component_flags` array. Indien dit het geval is, valt de pixel binnen de kleurrange en kan deze net zoals in versie 3 de waarde op de overeenkomstige rij- en kolom-index ophogen in de twee vectoren, die het aantal pixels dat voldoet per rij en per kolom bijhouden.

Merk op dat in versie 4, in tegenstelling tot alle voorgaande versies, er gebruik gemaakt kan worden van coalesced memory acces. Iedere thread behandelt één kleurcomponent en iedere kleurcomponent zit opgeslagen in opeenvolgende memory locaties. Ieder thread werkt dus met data vanop opeenvolgende geheugenlocaties waardoor deze kleurcomponenten in één keer opgehaald kunnen worden. In eerdere versies werkte iedere thread ofwel met pixels in zijn geheel of met rijen/kolommen van pixels. Er werd dus striding toegepast, maar deze striding verhindert de mogelijkheid tot coalesced memory access [3].

De relevante delen van versie 4 kunnen gevonden worden in Appendix G.

4.7 Thread divergence

In versie 5 werd er opnieuw voor iedere pixel een individuele thread voorzien maar werd er een optimalisatie geïmplementeerd die in de C++ versie ook aanwezig was. Vanaf het moment dat een thread opmerkt dat één van de kleurcomponenten, horende bij de pixel die deze thread controleert, buiten de geldige range valt stopt deze thread onmiddellijk met uitvoeren en wordt er verder niet meer gekeken naar de andere kleurcomponenten. Deze implementatie/optimalisatie introduceert thread divergence: sommige threads zullen alle drie de checks doorlopen, andere threads zullen na de 1ste check al stoppen. We introduceren hier dus een optimalisatie ten koste van een mogelijke performance daling. Deze keuze kunnen we echter verantwoorden door te stellen dat de thread divergence enkel optreedt in de buurt van het object, waar pixels van twee verschillende gebieden voorkomen (object- en achtergrondgebied). In dat gebied zullen sommige threads na de eerste controle al beëindigen, terwijl andere de volledige kernelfunctie doorlopen. Wanneer we in het midden van het object zitten of ergens in de achtergrond komt er weinig thread divergence voor, alle threads zullen namelijk een gelijkaardige zone bekijken. Hoe vaak thread divergence effectief optreedt is dus vrij beperkt en ook vrij "lokaal" in het frame. Waardoor de daling in performance relatief beperkt zal zijn. De verdere werking van deze versie is volledig analoog aan voorgaande versies. De relevante delen van deze implementatie kunnen gevonden worden in Appendix H.

4.8 Asynchronous operation

In de laatste versie wordt terwijl de ene frame verwerkt wordt, het volgende frame al ingeladen. De originele doelstelling was dat we hier graag de verwerking van het resultaat van de GPU kernel doen (het zoeken van het midden van de bol op basis van de inhoud van de twee vectoren die het aantal pixels dat voldoet per rij en per kolom bijhouden) en ondertussen het volgende frame analyseren. Dit bleek echter niet mogelijk te zijn. Bij asynhronous operation kan de CPU verder uitvoeren tijdens het uitvoeren van de CUDA-kernel op de GPU. Echter moet de CPU wachten tot de resultaten van de GPU beschikbaar zijn om deze verder te kunnen verwerken, we gebruiken hiervoor `cudaDeviceSynchronize()`. Deze functie zorgt ervoor dat de CPU wacht en dus eigenlijk geen verder werk meer doet. We hadden een gelijkaardig probleem bij het asynchroon inladen van de volgende image: aangezien de frames verwerkt worden via een `for`-loop zal de CPU omwille van de `cudaDeviceSynchronize()` niet verder uitvoeren en zal de volgende iteratie waarin de image ingeladen wordt dus niet starten tot de vorige frame volledig geanalyseerd is door de GPU. Als work-around hiervoor hebben we telkens de image voor iteratie `i + 1` ingeladen op het einde van iteratie `i`. In de eerste "iteratie" wordt dan enkel de image ingeladen (de kernelfunctie wordt overgeslagen, er is nog geen image ingeladen) en in de volgende iteratie wordt de kernel effectief uitgevoerd. De relevante delen van versie 6 kunnen gevonden worden in Appendix I.

5 Observaties

5.1 Dubbele reductie

In *Figuur 6* is te zien dat de verwerkingstijd per frame voor versie 1 (de CUDA-implementatie met reductie) iets hoger ligt dan die voor versie 0 (de CUDA-implementatie zonder reductie). De gemeten tijd omvat de volledige uitvoeringstijd van de CPU-functie.Dit betekent dat zowel geheugenallocaties, het overbrengen van data naar de GPU, de uitvoering van de kernel, als het terug te kopiëren van de resultaten naar de CPU zijn inbegrepen.

De keuze om deze totale uitvoeringstijd te meten is gebaseerd op het feit dat de berekening van het midden van het object in versie 0 volledig op de CPU gebeurt, terwijl deze in versie 1 gedeeltelijk naar de GPU is verplaatst. In versie 1 worden de minimum- en maximumindices op de GPU bepaald via dubbele reductie, waarna het gemiddelde nog op de CPU wordt berekend. Om deze twee implementaties te vergelijken hadden we bij beide implementaties ook de tijd nodig om het midden te bepalen en niet enkel de tijd in de CUDA-kernels.

Enerzijds komt dit resultaat niet overeen met onze verwachtingen, aangezien versie 1 meer gebruikmaakt van parallelle verwerking via een min-max-reductie op de GPU. Anderzijds brengt deze aanpak extra implementatiecomplexiteit met zich mee. Hierdoor is het mogelijk dat de eenvoudige lineaire iteratie over de array op de CPU in versie 0 uiteindelijk efficiënter blijkt te zijn dan de parallelle verwerking in versie 1.

Daarnaast vereist de reductie dat de grootte van de array een macht van twee is. In onze use case betekent dit bijvoorbeeld dat voor de beeldbreedte een array van 2048 elementen wordt gebruikt, terwijl de CPU slechts 1280 elementen moet verwerken.

Tot slot introduceert versie 1 ook bijkomende CPU-operaties. Vooraleer de reductie opgestart kan worden, moet immers eerst de kleinste macht van twee groter dan de oorspronkelijke breedte en hoogte van het frame bepaald worden. Ook deze stap draagt bij aan de hogere totale verwerkingstijd van versie 1. Dit zou wel opgelost kunnen worden door deze bewerking slechts één keer uit te voeren in plaats van voor elk frame of deze waarde als constante aan het begin van het programma te definiëren.

In *Figuur 7* wordt de totale uitvoertijd van beide versies weergegeven voor de volledige video, bestaande uit 150 frames. Het programma werd twintig keer uitgevoerd om een betrouwbare meting te bekomen. Op basis van deze herhaalde metingen werd een boxplot opgesteld, zodat de resultaten niet beïnvloed worden door individuele uitschieters of toevallige veranderingen in de uitvoeringstijd.

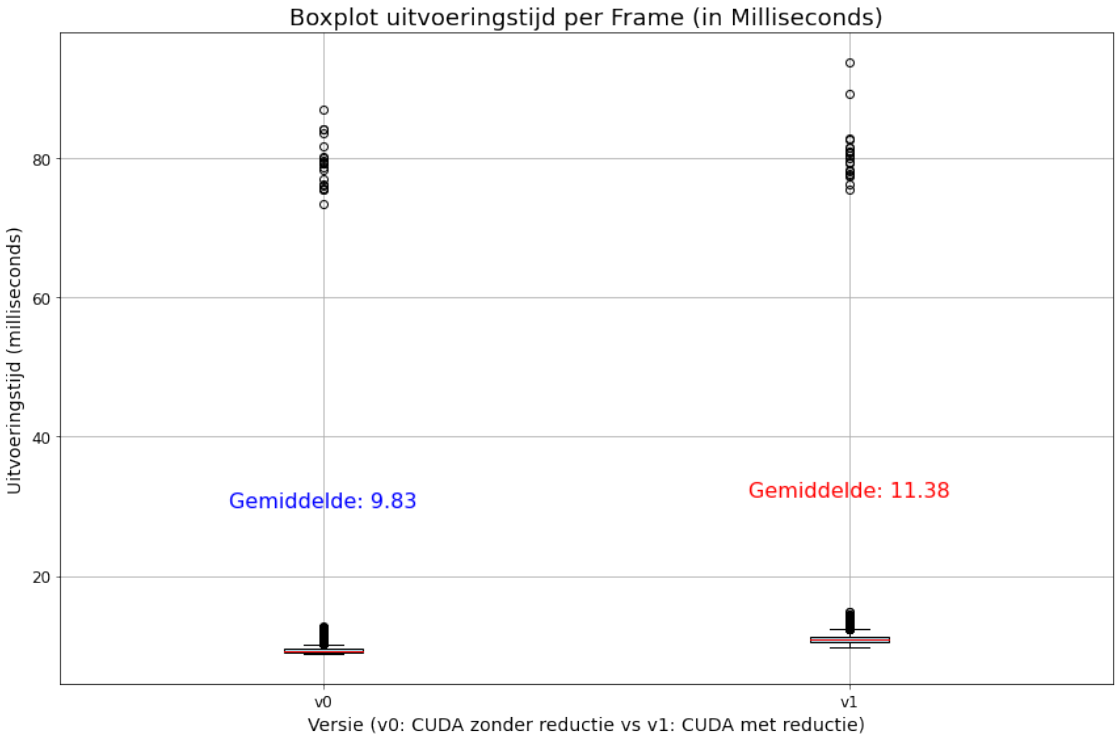


Figure 6: Boxplot uitvoeringstijd per frame (v0 en v1)

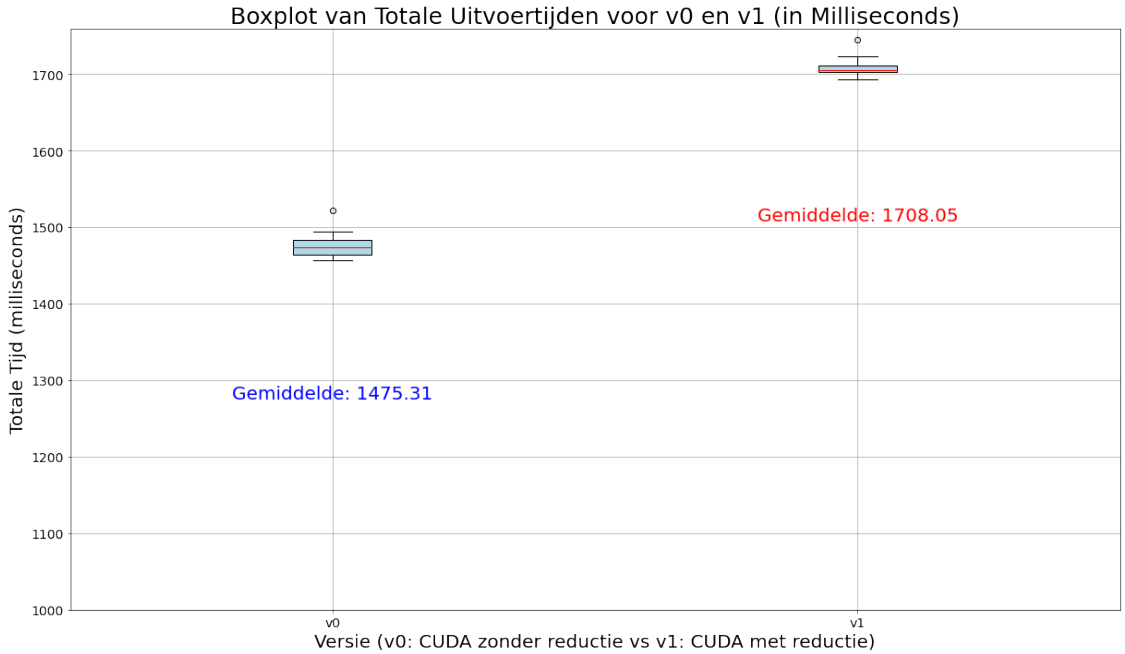


Figure 7: Boxplot totale uitvoeringstijd voor v0 en v1 (video van 150 frames)

5.2 Warp efficiency

In *Figuur 8* en *Figuur 9* zien we duidelijk dat de verwerkingstijd per frame en de verwerkingstijd in totaal (voor 150 frames) bijna dubbel zo hoog ligt wanneer we gebruik maken van telkens één thread per warp in vergelijking met wanneer we volledig gevulde warps gebruiken. Dit is logisch want threads worden gescheduled in warps. Wanneer een warp gescheduled wordt en er voert telkens maar één thread van deze warp effectief nuttig werk uit, benutten we telkens maar 1/32ste van de effectieve rekenkracht van deze warp wat ervoor zorgt dat GPU-resources verspild worden aan threads die geen nuttig werk uitvoeren. Dit leidt tot de verlaagde performance die we zien.

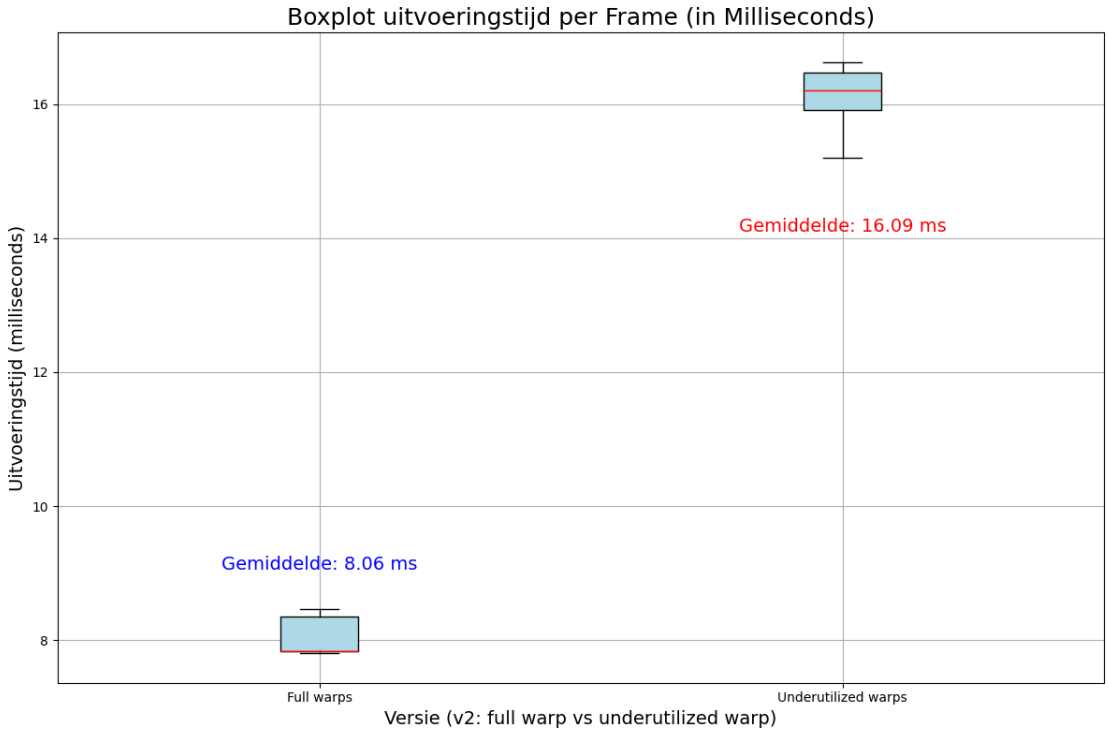


Figure 8: Boxplot uitvoeringstijd per frame voor V2 met full warps en underutilized warps

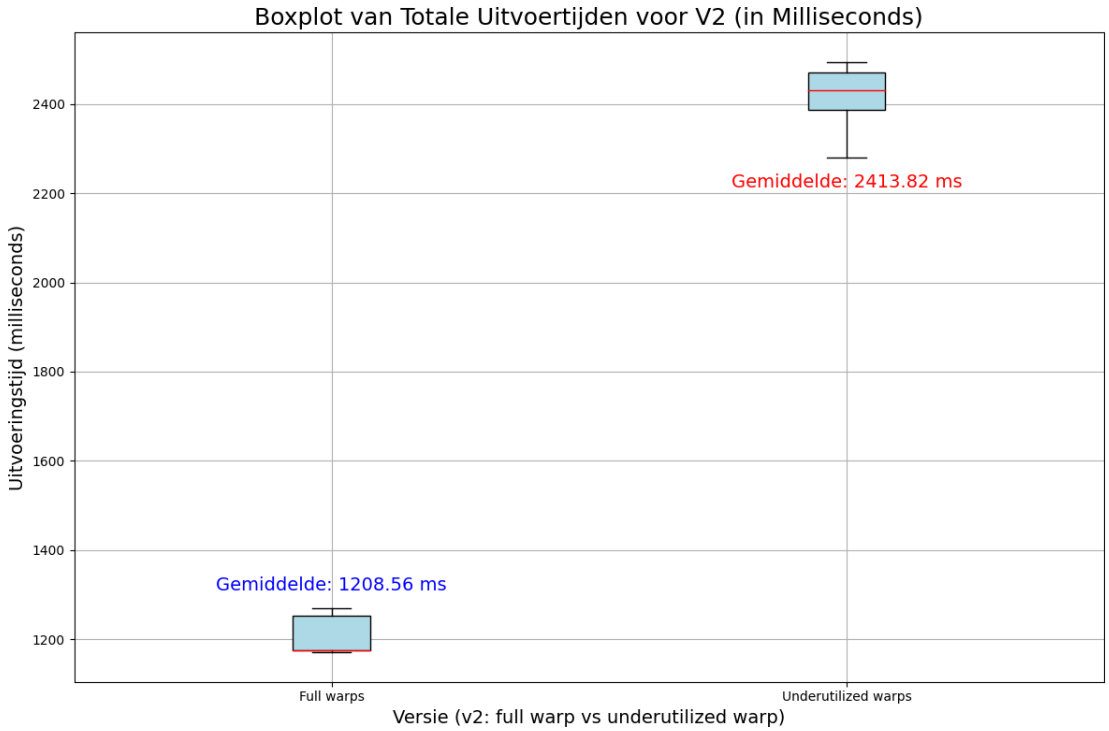


Figure 9: Boxplot totale uitvoeringstijd voor V2 met full warps en underutilized warps (video van 150 frames)

5.3 "Overclocking" the GPU

Om de performance van versies 3 en 4 te evalueren, vergelijken we de totale verwerkingstijd en de verwerkingstijd per frame tussen deze versies onderling en met versie 0. Het resultaat is te zien in *Figuur 10* en *Figuur 11*. We zien duidelijk dat versie 3 hier de meest optimale versie is: zowel de totale verwerkingstijd als de verwerkingstijd per frame is voor deze versie het laagst. Als we er opnieuw van uitgaan dat we werken met een GT 740, kunnen we 2048 threads effectief in parallel uitvoeren. In versie 0 worden er maar 2000 threads voorzien waardoor er telkens 48 threads onbenut blijven tijdens iedere stream multiprocessor-cycle. In de andere twee versies worden telkens wel het maximum aantal threads, dat parallel uitgevoerd kan worden, uitgevoerd. Dit zorgt al voor een eerste verklaring waarom versie 3 en 4 sneller zijn dan versie 0.

Daarnaast wordt iedere pixel in versie 0 twee keer verwerkt: één keer door een thread die de hele kolom verwerkt en één keer door een andere thread die de volledige rij verwerkt. Dit leidt ook tot een langere uitvoeringstijd.

Tot slot zorgt ook de manier waarop de pixels verwerkt worden voor een verschil in performance. Stel, een thread heeft `x` tijdseenheden nodig om één pixel te verwerken. In dit geval zullen er in versie 0 `1280 * x` tijdseenheden nodig zijn om het frame te verwerken: alle threads kunnen in parallel uitvoeren en iedere tijdseenheid verwerkt iedere thread een pixel. In het geval dat aan iedere pixel een thread wordt toegekend (versie 3) hebben we 921600 threads waarvan er telkens 2048 in parallel kunnen uitvoeren, waardoor we 450 iteraties nodig hebben. Echter, iedere iteratie verwerkt iedere thread één pixel en heeft dus een "uitvoeringstijd" van `x`, wat een totale verwerkingstijd van `450 * x` per frame oplevert, wat veel lager is dan de verwerkingstijd (per frame) in versie 0.

De reden dat versie 4 slechter presteert dan versie 3 heeft vermoedelijk te maken met de extra complexiteit die versie 4 nodig heeft om correct te werken. Het synchroniseren van de threads om dan in de `component_flags` array te gaan checken of de voorgaande componenten ook binnen de kleurenrange vallen introduceert enige overhead. Daarnaast zorgt het hergebruiken van de thread die de rode kleurcomponent verwerkte om te checken of de andere kleurcomponenten voldoen, voor thread divergence binnen een warp, wat ook de performance licht doet dalen. Anderzijds maakt versie 4 wel gebruik van coalesced memory access zoals besproken in de *Methodology*-sectie. De boost in performance die dit met zich meebrengt weegt echter niet op tegen de zojuist vermelde effecten die de performance iets doen dalen.

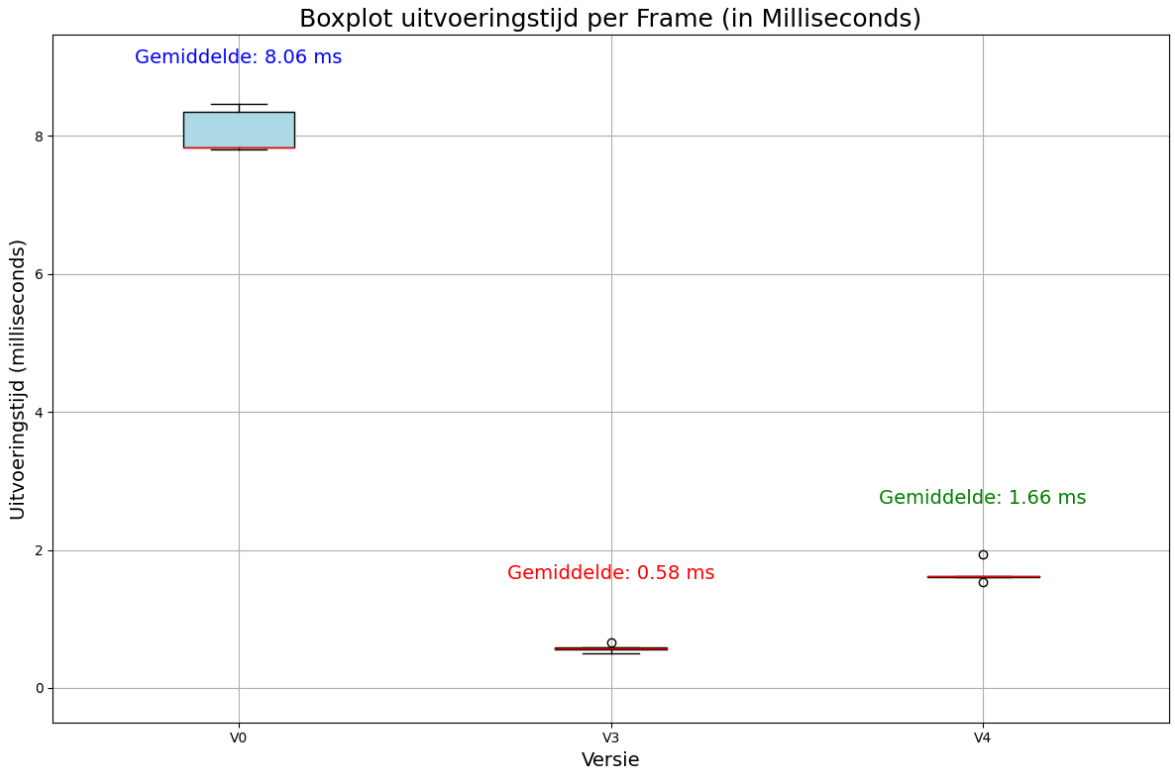


Figure 10: Boxplot uitvoeringstijd per frame voor V0, V3 en V4

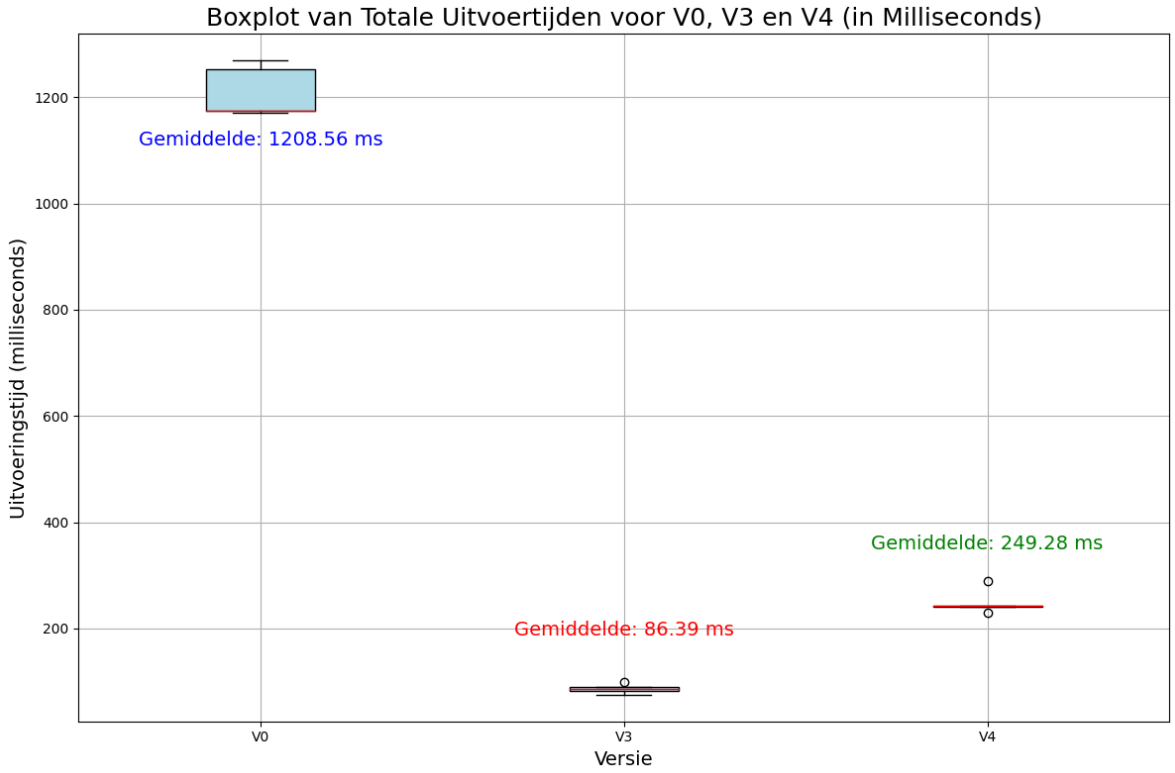


Figure 11: Boxplot totale uitvoeringstijd voor V0, V3 en V4 (video van 150 frames)

5.4 Thread divergence

Wanneer we effectief naar de performance van versie 5 kijken en dit vergelijken met versie 3 (zie *Figuur 12* en *Figuur 13*) (hun werking is namelijk volledig analoog alleen wordt er in versie 5 vroegtijdig gestopt wanneer één kleurcomponent mismatched) zien we dat de performance van versie 3 en 5 zeer dicht bij elkaar in de buurt liggen. We

kunnen echter wel opmerken dat over het algemeen (zowel qua totale verwerkingstijd als verwerkingstijd per frame) versie 5 net iets trager is dan versie 3. We zien dus wel degelijk het effect van thread divergence op de performance.

Dit brengt ons tevens tot reflectie over het toepassen van optimalisaties. Het is duidelijk dat het vroegtijdig stoppen na een mismatch van een kleurcomponent op de CPU een snelheidswinst zou opleveren. Maar dezelfde optimalisatie geeft in een sterk geparalleliseerde omgeving juist een tragere verwerkingstijd. We moeten dus eventuele "optimalisaties" aanpassen aan de hardware waarop deze uitgevoerd worden.

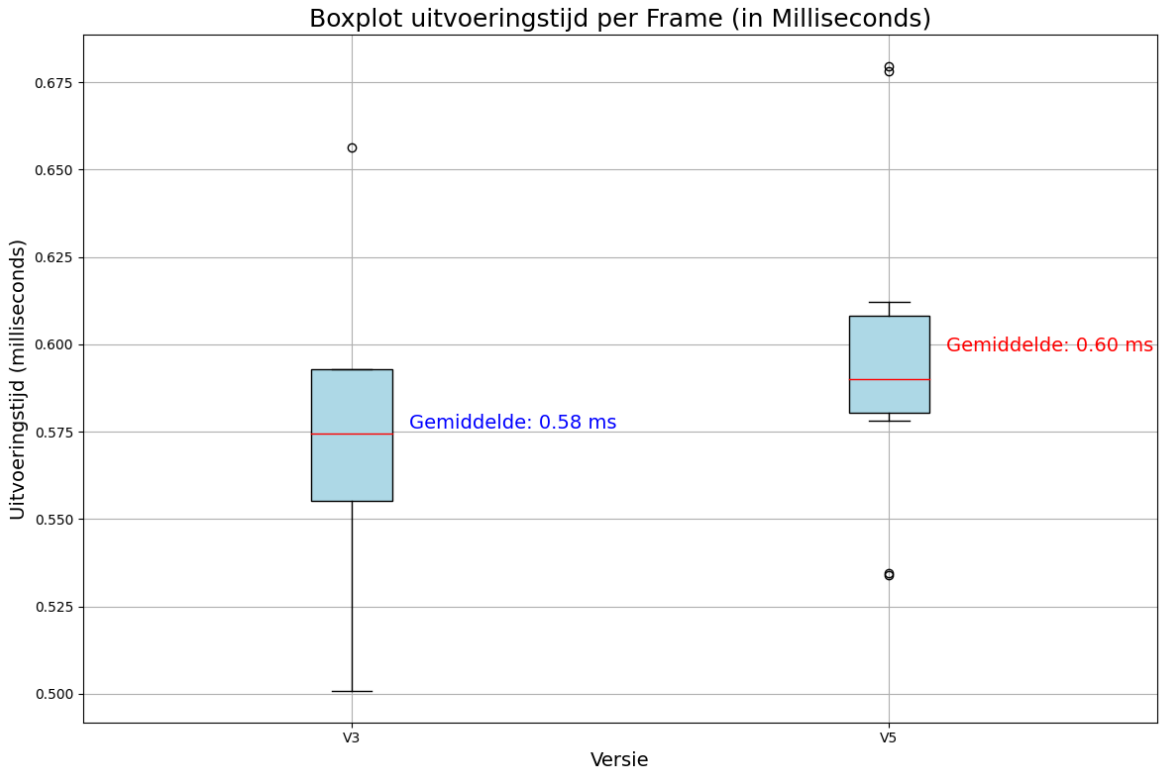


Figure 12: Boxplot uitvoeringstijd per frame voor V5 en V3

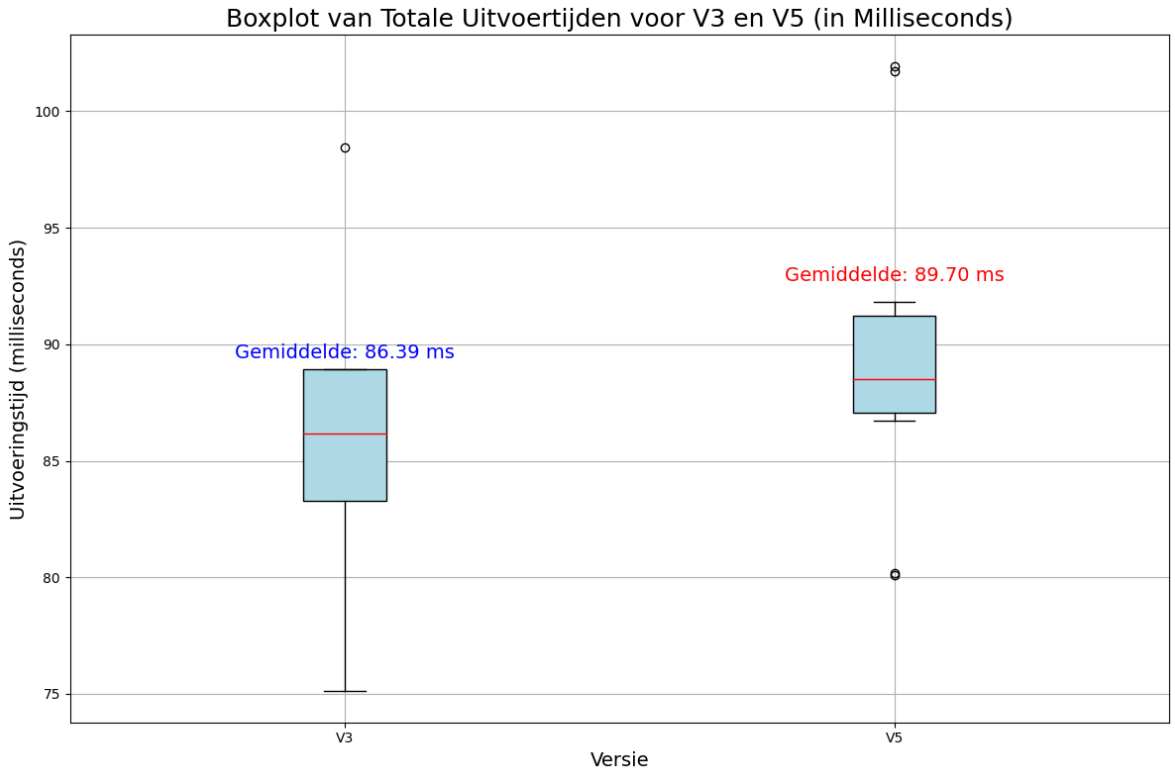


Figure 13: Boxplot totale uitvoeringstijd voor V5 en V3 (video van 150 frames)

5.5 Asynchronous operation

Om te kijken of het asynchroon inladen van de volgende frame terwijl de GPU de huidige frame analyseert effectief een performance boost geeft, vergelijken we versie 6 en versie 0. Hier wordt enkel de totale verwerkingstijd vergeleken aangezien we minstens twee iteraties nodig hebben om het ene frame te analyseren en vervolgens het andere in te laden. Het resultaat is te zien in *Figuur 14*.

Het resultaat hier gaat tegen onze verwachtingen in. We zouden namelijk verwachten dat het volgende frame parallel inladen tijdens het analyseren van het huidige frame een snelheidswinst zou opleveren maar dit is niet het geval. In tegendeel, we hebben een lichte performance daling. We kunnen dit echter wel als volgt verklaren: we vermoeden dat er een kleine fout in versie 6 zit. Er wordt namelijk in de `for`-loop `cudaStreamDestroy()` gecalled. In de documentatie staat het volgende: "*In case the device is still doing work in the stream when cudaStreamDestroy() is called, the function will return immediately and the resources associated with the stream will be released automatically once the device has completed all work in the stream.*" [1]. De device moet dus wachten tot al het werk in de stream voltooid is vóór de stream vrijgegeven kan worden, waardoor we eigenlijk niet met asynchrone operatie te maken hebben. We hebben echter wel extra overhead gecreëerd met de `if`-checks. Dit verklaart waarom versie 6 effectief trager is dan versie 0.

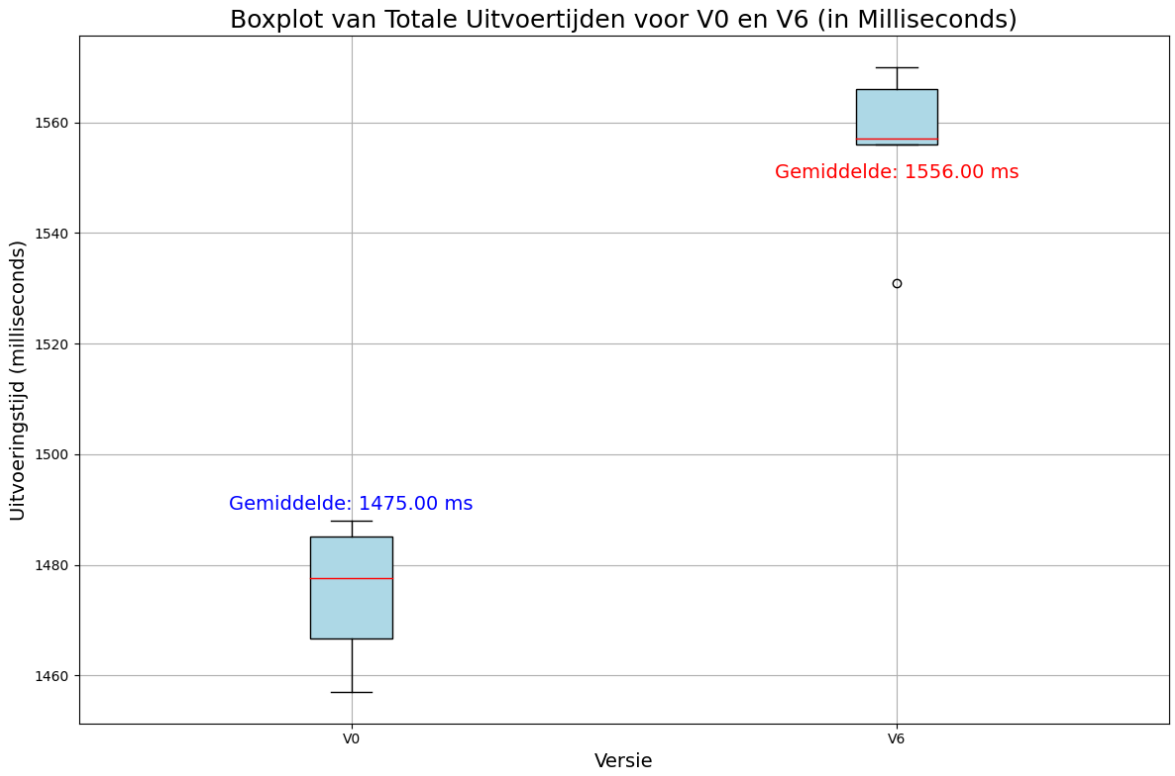


Figure 14: Boxplot totale uitvoeringstijd voor V0 en V6 (video van 150 frames)

5.6 Overzicht van de verschillende versies

In *Figuur 15* wordt een overzicht gegeven van de tijd die nodig is om één frame te verwerken. De waarden in deze figuur zijn gebaseerd op het gemiddelde van meerdere runs. De grafiek is opgedeeld in twee delen: aan de linkerkant worden de totale uitvoeringstijden (met allocaties, kopiëren van data naar de GPU...) per frame weergegeven voor versie 0 en versie 1, terwijl aan de rechterkant de gemiddelde tijd wordt getoond die per frame in de kernel wordt gespendeerd.

De individuele resultaten en verschillen tussen de versies zijn eerder in dit verslag besproken. Deze figuur biedt een samenvattend overzicht, waaruit duidelijk blijkt dat versie 3 en versie 5 de beste prestaties leveren.

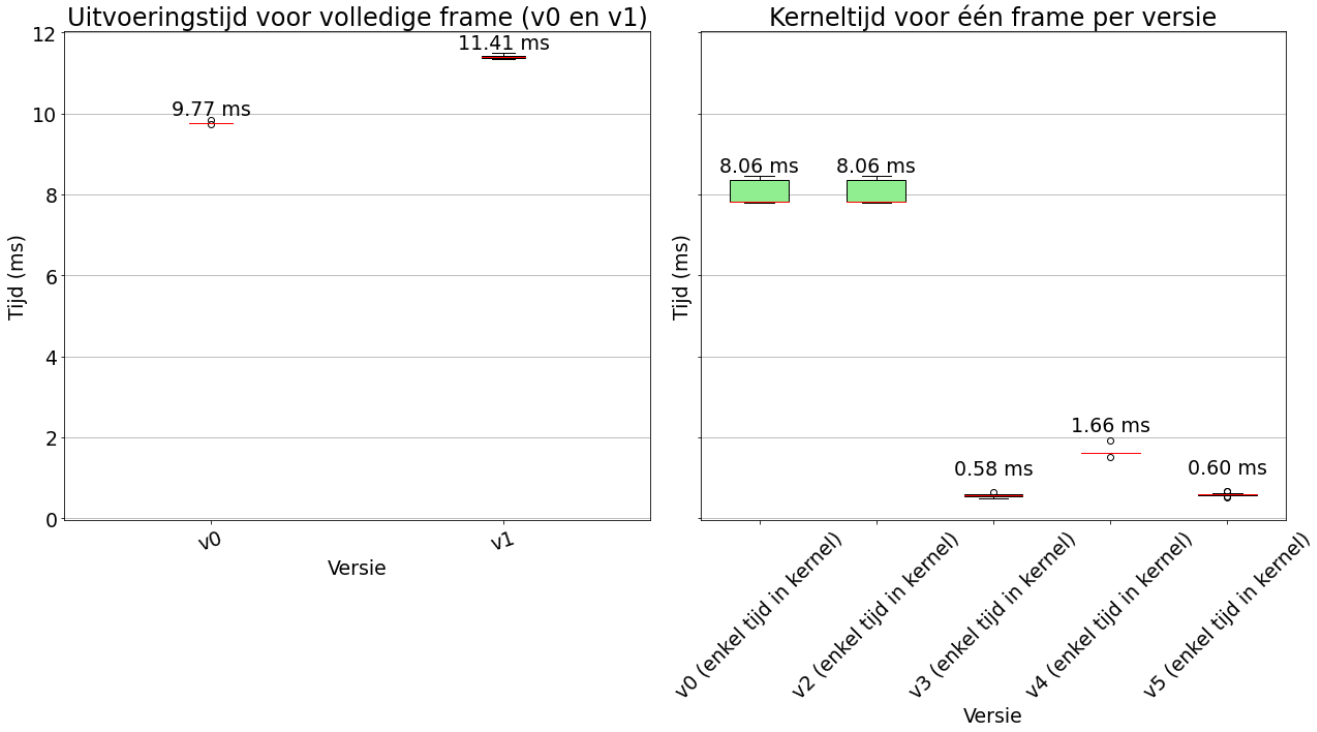


Figure 15: Vergelijking frame-tijden per versie

5.7 CPU programming VS GPU programming

Tot slot vergelijken we nog graag even een implementatie op de CPU met de meest optimale implementatie op de GPU. De resultaten op *Figuur 16* zijn per frame omdat, indien we naar een real-time processing systeem zouden willen gaan, hier ook de frames één voor één binnenkomen en zo snel mogelijk verwerkt moeten kunnen worden. We zien duidelijk dat de verwerkingstijd op de GPU ongeveer 10 keer lager is dan op de CPU. Dit toont aan dat deze use case uitermate geschikt is om uit te voeren op de GPU.

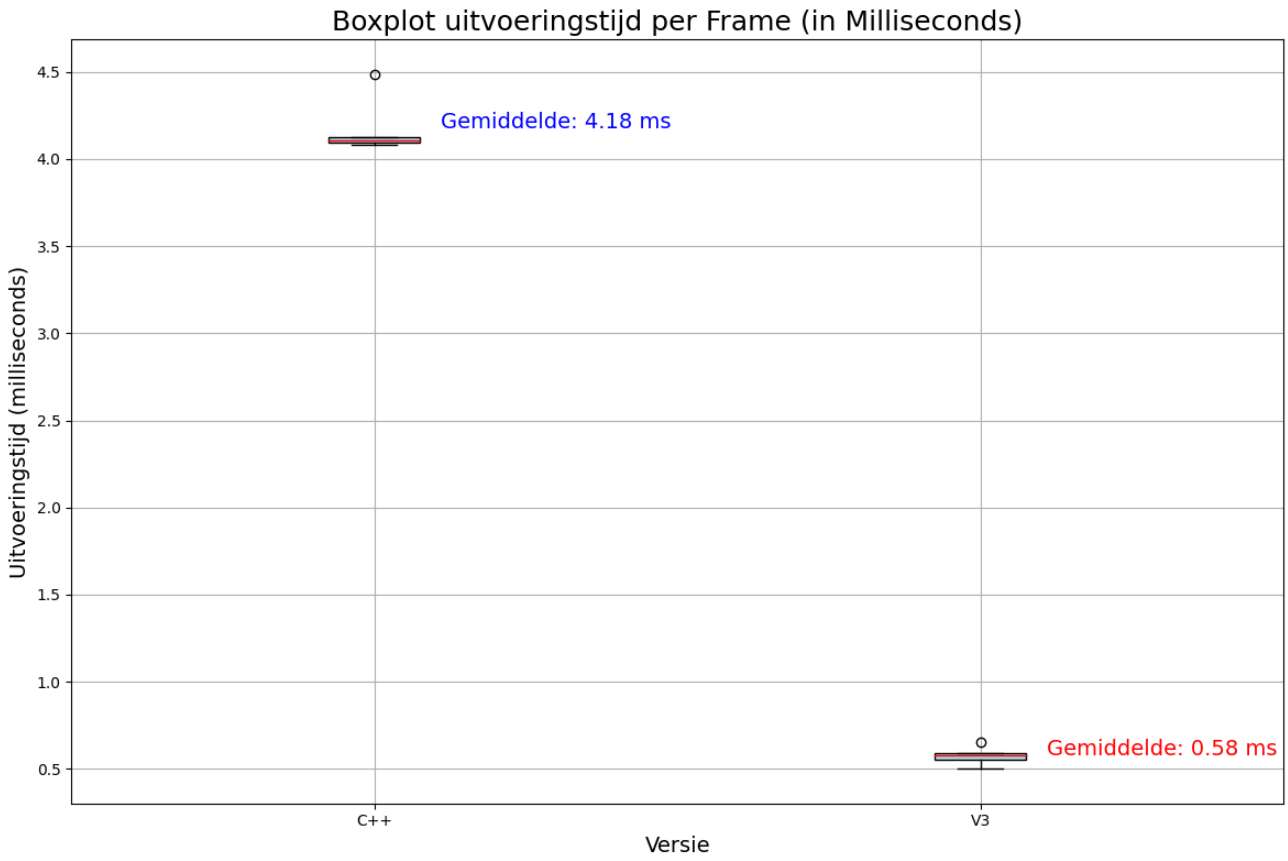


Figure 16: Vergelijking frame-tijden C++ VS. V3

6 Tekortkomingen en mogelijke verbeteringen

Hoewel de huidige implementatie functioneel is, zijn er nog een reeks mogelijke verbeteringen die in de toekomst aangepakt kunnen worden.

Een duidelijke tekortkoming is te merken bij versie 6 van onze implementatie. Een herwerking van deze versie om toch asynchrone operatie te voorzien kan interessant zijn voor een verder performance boost. Dit zou eenvoudig gedaan kunnen worden door enkel het 1ste frame in te laden op de plaats waar dit nu gebeurd en alle andere frames in te laden vóór de stream vrijgegeven wordt.

Daarnaast hebben we ook nog gekeken naar mogelijkheden om te experimenteren met de verschillende soorten geheugen, maar het enige type geheugen waar we gebruik van konden maken was het globaal geheugen: de frames waren namelijk te groot om ze in te kunnen laden in andere, sneller vormen van geheugen (zoals shared memory). Indien we echter eerst de frames zouden downscalen (zoals verder wordt besproken) zouden we eventueel wel nog gebruik kunnen maken van alternatieve vormen van geheugen. Vervolgens wordt bij elk frame de volledige afbeelding opnieuw geanalyseerd, zonder rekening te houden met de positie van het object in het vorige frame. In realistische toepassingen kan dit leiden tot onnodige rekenlast. Door de vorige positie van het object als uitgangspunt te nemen, zou men het zoekgebied in volgende frames sterk kunnen beperken, wat resulteert in kortere verwerkingstijden.

Een andere mogelijke verbetering betreft het gebruik van een hiërarchische resolutiebenadering. Hierbij zou men eerst de afbeelding op een lagere resolutie analyseren om globaal te bepalen in welk gebied op de afbeelding het object zich bevindt. Vervolgens zou men dit specifieke gebied op een hogere resolutie opnieuw analyseren, herhaaldelijk totdat men de originele resolutie of de gewenste precisie voor de toepassing heeft bereikt.

Een volgend belangrijk aandachtspunt is de efficiëntie van de gegevensoverdracht tussen de GPU en de CPU. In de huidige implementatie wordt de volledige array met gedetecteerde indexwaarden teruggestuurd naar de CPU, terwijl in werkelijkheid enkel de minimale en maximale waarde nodig zijn om het midden van het object te bepalen. Door uitsluitend deze twee waarden terug te kopiëren, kan de overhead van de geheugenoverdracht aanzienlijk worden verminderd. Dit kan gerealiseerd worden door, na de synchronisatie binnen de kernel, enkel

thread 0 de minimale en maximale waarde in gedeelde geheugenruimte of globale variabelen te laten schrijven. Hierdoor zijn geen atomic-operaties nodig.

Tot slot vereist de huidige kernel-implementatie dat de grootte van de inputarrays een macht van twee is. Dit leidt ertoe dat, bijvoorbeeld voor een beeldbreedte van 1280 pixels, gewerkt wordt met arrays van 2048 elementen. Dit introduceert zowel extra geheugenverbruik als onnodige bewerkingen. Een waardevolle uitbreiding zou een kernel zijn die ook efficiënt omgaat met arrays waarvan de grootte geen macht van twee is.

7 Conclusie

In dit project onderzochten we verschillende methoden om de baan van een object in een video te volgen, met een focus op GPU-versnelling via CUDA. We begonnen met een seriële implementatie in Python en C++, en schakelden vervolgens over op parallelle uitvoeringen op de GPU. We ontwikkelden zes verschillende versies van de implementatie met als doel een zo snel mogelijk resultaat te krijgen, terwijl we tegelijk een grondige analyse uitvoeren van verbeteringen. Uiteindelijk, behaalden we een resultaat van 0.58 ms per frame in de kernel. Ter vergelijking, het menselijk oog kan gemiddeld 30 tot 60 frames per seconde zien [2]. Dit komt overeen met ongeveer 16 milliseconden per frame. Onze resultaten zitten hier ruim onder, waardoor we deze als real-time kunnen beschouwen.

8 Link naar de code

De belangrijkste delen van de code zijn te vinden in de `Appendix`, zoals verwezen in dit verslag. De volledige code is ook te vinden op GitHub via de link `https://github.com/Indra2003c/Project_Advanced_Computerarchitectures`.

Appendix A Python implementatie (verkort)

```python
#Volledig code: zie github track_color.py

def in_range(pixel):
    R_value = pixel[0]
    G_value = pixel[1]
    B_value = pixel[2]
    if (R_lower <= R_value <= R_upper) and (G_lower <= G_value <= G_upper) and (B_lower <=
        B_value <= B_upper):
        return True
    else:
        return False

for frameNr in range(150):
    path = "./../frames/moving_ball_frames_png/" + str(frameNr) + ".png"
    # Laad de afbeelding
    frame = Image.open(path).convert("RGB")

    # Converteer naar een 3D NumPy-array (hoogte, breedte, 3 kleuren)
    rgb_values = np.array(frame)

    for i in range(720):
        for j in range(1280):
            if in_range(rgb_values[i][j]):
                if i > Row_upper_bound:
                    Row_upper_bound = i
                elif i < Row_lower_bound or Row_lower_bound == -1:
                    Row_lower_bound = i

                if j > Column_upper_bound:
                    Column_upper_bound = j
                elif j < Column_lower_bound or Column_lower_bound == -1:
                    Column_lower_bound = j

    Middle_row = math.ceil(((Row_upper_bound - Row_lower_bound)/2) + Row_lower_bound)
    Middle_column = math.ceil(((Column_upper_bound - Column_lower_bound)/2) + Column_lower_bound)

    with open("./../output/coordinates.txt", "a") as bestand:
        bestand.write(str(Middle_row) + " " + str(Middle_column) + "\n")
```

Appendix B C++ implementatie (verkort)

```cpp
#Volledige code: github Object_trajectory_array.cpp

int getRGBFlattenedIndex(int i, int j) {
    return i * W * C + j * C;
}

int main() {
    std::vector<std::pair<int, int>> object_coordinates;

    for (int frameNr = 0; frameNr < 150; frameNr++) {
        string filePath = "./../frames/moving_ball_frames_ppm/" + to_string(frameNr) + ".ppm";
        uint8_t* pixels = get_image_array_ppm(filePath.c_str());

        std::vector<int> find_row(H, 0); //initialiseert waarden op 0
        std::vector<int> find_column(W, 0);

        for (int i = 0; i < H; i++) {
            for (int j = 0; j < W; j++) {
                int index = getRGBFlattenedIndex(i, j);
                int r = (int)pixels[index];
                if (r >= R_lower && r <= R_upper) {
                    int g = (int)pixels[index + 1];
                    if (g >= G_lower && g <= G_upper) {
                        int b = (int)pixels[index + 2];
                        if (b >= B_lower && b <= B_upper) {
                            find_column[j] += 1;
                            find_row[i] += 1;
                        }
                    }
                }
            }
        }

        double sum_rows = 0, count_rows = 0;
        for (int i = 0; i < H; i++) {
            if (find_row[i] > 0) {
                sum_rows += i;
                count_rows += 1;
            }
        }
        double mean_row = (count_rows > 0) ? sum_rows / count_rows : 0;
        int roundedMeanRow = round(mean_row);

        double sum_columns = 0, count_columns = 0;
        for (int j = 0; j < W; j++) {
            if (find_column[j] > 0) {
                sum_columns += j;
                count_columns += 1;
            }
        }
        double mean_column = (count_columns > 0) ? sum_columns / count_columns : 0;
        int roundedMeanColumn = round(mean_column);

        object_coordinates.push_back({roundedMeanRow , roundedMeanColumn});
        free(pixels);
    }

    save_coordinates_to_file(object_coordinates);
    return 0;
}
```

Appendix C Implementatie CUDA v0: basisimplementatie op de CPU

```

#Volledige code: github OT_GPU_V0.cu


__global__ void find_object_colour(const uint8_t* pixels, int* row_counts, int* col_counts) {
    //Doordat we met een twee arrays werken, 1 voor de kolommen en 1 voor de rijen
    //en er 1 thread is per rij en kolom is er geen atomic operation nodig
    //elk element in de array kan namelijk maar door 1 thread aangepast worden

    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if(i < H){
        //eerste H threads zijn voor rijen
        int count = 0;
        for (int j = 0; j < W; ++j) {
            int idx = i * W * C + j * C;
            uint8_t r = pixels[idx];
            uint8_t g = pixels[idx + 1];
            uint8_t b = pixels[idx + 2];

            if (r >= R_lower && r <= R_upper &&
                g >= G_lower && g <= G_upper &&
                b >= B_lower && b <= B_upper) {
                count++;

            }
        }
        row_counts[i] = count;
    }
    else if(i < (H+W)){
        int j = i -  H;
        //volende W threads zijn voor kolommen
        int count = 0;
        for (int c = 0; c < H; ++c) {
            int idx = c * W * C + j * C;
            uint8_t r = pixels[idx];
            uint8_t g = pixels[idx + 1];
            uint8_t b = pixels[idx + 2];

            if (r >= R_lower && r <= R_upper &&
                g >= G_lower && g <= G_upper &&
                b >= B_lower && b <= B_upper) {
                count++;
            }
        }
        col_counts[j] = count;
    }
    else{
        return;
    }

}

int main() {
    std::vector<std::pair<int, int>> object_coordinates;

    uint8_t* d_pixels;
    int* d_row_counts;
    int* d_col_counts;

    for (int frameNr = 0; frameNr < 150; ++frameNr) {
        std::string path = "./CUDA_versions/frames/moving_ball_frames_ppm/" + std::to_string(
            frameNr) + ".ppm";
        uint8_t* pixels = get_image_array_ppm(path.c_str());

        std::vector<int> row_counts(H, 0);
        std::vector<int> col_counts(W, 0);

        cudaMalloc(&d_pixels, H * W * C);
        cudaMemcpy(d_pixels, pixels, H * W * C, cudaMemcpyHostToDevice);

        cudaMalloc(&d_row_counts, H * sizeof(int));
        cudaMalloc(&d_col_counts, W * sizeof(int));
        cudaMemset(d_row_counts, 0, H * sizeof(int));
        cudaMemset(d_col_counts, 0, W * sizeof(int));
```

```
75
76          // kernel gegevens
77          int row_block_size = 1024;
78          int col_block_size = 1024;
79          int grid_size = ((H + W) / row_block_size) +1;
80
81          find_object_colour<<<grid_size, row_block_size>>>(d_pixels, d_row_counts, d_col_counts);
82          cudaDeviceSynchronize();
83
84          cudaMemcpy(row_counts.data(), d_row_counts, H * sizeof(int), cudaMemcpyDeviceToHost);
85          cudaMemcpy(col_counts.data(), d_col_counts, W * sizeof(int), cudaMemcpyDeviceToHost);
86
87          //Dit wnr reductie is ook op GPU doen, nu op CPU
88          //kolom/rijnummer in array zetten en op een manier midden zoeken
89          double row_sum = 0, row_count = 0, col_sum = 0, col_count = 0;
90          for (int i = 0; i < H; ++i) {
91              if (row_counts[i]) { row_sum += i; row_count += 1; }
92          }
93          for (int j = 0; j < W; ++j) {
94              if (col_counts[j]) { col_sum += j; col_count += 1; }
95          }
96
97          int row_mean = (row_count > 0) ? round(row_sum / row_count) : 0;
98          int col_mean = (col_count > 0) ? round(col_sum / col_count) : 0;
99          object_coordinates.push_back({row_mean, col_mean});
100
101         free(pixels);
102         cudaFree(d_pixels);
103         cudaFree(d_row_counts);
104         cudaFree(d_col_counts);
105     }
106     save_coordinates_to_file(object_coordinates) ;
107     return 0;
108 }
```

Appendix D Implementatie CUDA v1: CUDA met reductie

```
#Volledige code: github OT_GPU_V1_reductie.cu

__global__ void find_object_colour(const uint8_t* pixels, int* row_counts, int* col_counts) {
    #analoog aan v0 zonder reductie
}

#dubbele reductie
__global__ void min_max_index(int* row_counts, int* col_counts, int log_2_H, int log_2_W, int
    H_power, int W_power) {

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int* d_arr;
    int log_2_size;
    int N;
    for(int i = 0; i <2; i++){
        if(i == 0){
            d_arr = row_counts;
            log_2_size = log_2_H;
            N = H_power;
        }
        else{
            d_arr = col_counts;
            log_2_size = log_2_W;
            N = W_power;

        }

        for(int iteratie = 1; iteratie <= log_2_size; iteratie++){
            //enkel threads die nodig zijn voor die iteratie gebruiken

            //Threads moeten min steken in val1 en max in val2
            if(iteratie == 1 && idx < pow(2, log_2_size)/(pow(2,iteratie))){

                int val1 = idx*pow(2,iteratie) ;
                int val2 = idx*pow(2,iteratie) + (pow(2,iteratie)/2);

                if(d_arr[val1] == 0 && d_arr[val2] !=0 ){
                    //als 1 van de twee een nul is dan willen we het getal behouden
                    d_arr[val1] = d_arr[val2];
                }
                //pas ze aan zodat het kleinste van de twee links zit
                //dit is in principe overbodig aangezien de indices in de array staan en deze dus
                     geordend zijn
                else if(d_arr[val2] < d_arr[val1]){
                    int tmp = d_arr[val1];
                    d_arr[val1] = d_arr[val2];
                    d_arr[val2] = tmp;
                }
            }
            //Threads nodig voor minimum reductie
            //Verplaats het minimum naar links
            else if(idx < pow(2, log_2_size)/(pow(2,iteratie))){
                int val1 = idx*pow(2,iteratie) ;
                int val2 = idx*pow(2,iteratie) + (pow(2,(iteratie-1)));

                if(d_arr[val1] == 0 && d_arr[val2] !=0 ){
                    //als 1 van de twee 0 is dan willen we getal behouden
                    d_arr[val1] = d_arr[val2];
                }
                else if(d_arr[val2] == 0 && d_arr[val1] !=0 ){
                    d_arr[val2] = d_arr[val1];
                }
                else if(d_arr[val2] > d_arr[val1]){
                    d_arr[val2] = d_arr[val1];
                }

            }

            //Threads nodig voor maximum reductie naar rechts
            else if(idx >= pow(2, log_2_size)/(pow(2,iteratie)) && idx < 2* pow(2, log_2_size)/(
                pow(2,iteratie))){
                int new_idx =idx- pow(2, log_2_size)/(pow(2,iteratie));

                int val1 = N - 1 - (new_idx*pow(2,iteratie) + (pow(2,(iteratie-1)))) ;
                int val2 = N - 1 - new_idx*pow(2,iteratie); //max index, stays to the right
```

```
                if(d_arr[val2] < d_arr[val1]){
                    d_arr[val2] = d_arr[val1];
                }

            }

            __syncthreads();

        }

    }
}

int nextPowerOf(int x){
    return std::pow(2, std::ceil(std::log2(x)));
}
int main() {
    std::vector<std::pair<int, int>> object_coordinates;

    uint8_t* d_pixels;
    int* d_row_counts;
    int* d_col_counts;

    int H_power = nextPowerOf(H);
    int W_power = nextPowerOf(W);

    for (int frameNr = 0; frameNr < 150; ++frameNr) { //150
        std::string path = "./CUDA_versions/frames/moving_ball_frames_ppm/" + std::to_string(
            frameNr) + ".ppm";
        uint8_t* pixels = get_image_array_ppm(path.c_str());

        std::vector<int> row_counts(H_power, 0);
        std::vector<int> col_counts(W_power, 0);

        cudaMalloc(&d_pixels, H * W * C* sizeof(int));
        cudaMemcpy(d_pixels, pixels, H * W * C , cudaMemcpyHostToDevice);

        cudaMalloc(&d_row_counts, H_power * sizeof(int));
        cudaMalloc(&d_col_counts, W_power * sizeof(int));
        cudaMemset(d_row_counts, 0, H_power * sizeof(int));
        cudaMemset(d_col_counts, 0, W_power * sizeof(int));

        int row_block_size = 1024;
        int col_block_size = 1024; //1280
        int grid_size = ((H + W) / row_block_size) +1;

        find_object_colour<<<grid_size, row_block_size>>>(d_pixels, d_row_counts, d_col_counts);
        cudaDeviceSynchronize();
        grid_size = ((H_power + W_power)  / row_block_size) +1;
        min_max_index<<<grid_size, row_block_size>>>(d_row_counts, d_col_counts, log2(H_power),
            log2(W_power), H_power, W_power);
        cudaDeviceSynchronize();

        cudaMemcpy(row_counts.data(), d_row_counts, H_power * sizeof(int), cudaMemcpyDeviceToHost
            );
        cudaMemcpy(col_counts.data(), d_col_counts, W_power * sizeof(int), cudaMemcpyDeviceToHost
            );


        int n = row_counts.size()-1;

        int row_mean = round((row_counts[n]+ row_counts[0])/2);
        n = col_counts.size()-1;
        int col_mean = round((col_counts[n]+ col_counts[0])/2);
        object_coordinates.push_back({row_mean, col_mean});

        free(pixels);
        cudaFree(d_pixels);
        cudaFree(d_row_counts);
        cudaFree(d_col_counts);
    }
    save_coordinates_to_file(object_coordinates) ;
    return 0;
}
```

Appendix E Implementatie CUDA V2: full warps VS. underutilized warps

```
#Volledige code: github OT_GPU_V2.cu

find_object_colour<<<grid_size, row_block_size>>>(d_pixels, d_row_counts, d_col_counts);
//this one has two block with each 1024 threads => we have 64 warps and the penultimate one
//uses 16 out of it's 32 threads (the other 16 idle) and the last one uses none of it's 32
//threads (all of them idle)

find_object_colour<<<(H + W), 1>>>(d_pixels, d_row_counts, d_col_counts);
//this one uses 2000 blocks with each one thread (the other 31 threads of each block are idling)
//=> the GPU resources have been overprovisioned
```

Appendix F Implementatie CUDA V3: één thread per pixel

```
#Volledige code: github OT_GPU_V3.cu

//er word voor iedere pixel een thread voorzien => 720 * 1280 = 921.600 threads = overklokken
    scenario => threads moeten in groepjes sequentieel uitvoeren

__global__ void find_object_colour(const uint8_t* pixels, int* row_counts, int* col_counts) {

    int i = blockIdx.x * blockDim.x + threadIdx.x; //thread index

    if(i < W * H){
        int idx = i * C; //index waarnaar thread i moet kijken in de flattened image array
                         //  0        1        2        => thread index (i)  |
                         // [R, G, B, R, G, B, R, G, B]                       | => i * C (= 3) =
                             idx
                         //  0  1  2  3  4  5  6  7  8  => array index (idx) |

        int count = 0;

        uint8_t r = pixels[idx];
        uint8_t g = pixels[idx + 1];
        uint8_t b = pixels[idx + 2];

        if (r >= R_lower && r <= R_upper &&
            g >= G_lower && g <= G_upper &&
            b >= B_lower && b <= B_upper) {

            int row_index = i / W;
            int column_index = i % W;

            atomicAdd(&row_counts[row_index], 1);
            //we moeten hier een atomicAdd doen aangezien iedere thread nu 1 pixel
                vertegenwoordigt
            //=> meerdere threads accessen gelijktijdig dezelfde index in de row_counts array en
                de
            //col_counts array aangezien meerdere pixels op eenzelfde rij en kolom staan (deze
                arrays
            //hebben een lengte die gelijk is aan het aantal kolommen en rijen in de afbeelding)
            //
            //mogelijke andere manier:
            //1 flattened array maken met grote 921600 (voor iedere pixel een slot) en dan in
                preprocessing
            //de index in deze flat array terugrekenen naar een matrix en zo het gemiddelde
                bepalen door het
            //midden te tellen op de CPU te doen

            atomicAdd(&col_counts[column_index], 1);
        }
    }
}

int main() {
    ...

    int grid_size = ((H * W) / row_block_size); //levert 900 blocks op

    cudaEventRecord(start);
```

```
    find_object_colour<<<grid_size, row_block_size>>>(d_pixels, d_row_counts, d_col_counts); //
        wordt gecalled met 900 blocks van elk 1024 threads

    ...
}
```

Appendix G Implementatie CUDA V4: één thread per kleurcomponent

```
#Volledige code: github OT_GPU_V4-5.cu

//er word voor iedere kleurcomponent een thread voorzien => 720 * 1280 * 3 = 2.764.800 threads =
    extreme overklokken scenario => threads moeten in groepjes sequentieel uitvoeren
//merk ook op dat we hier wel gebruik kunnen maken van coalesced memory: alle kleurcomponenten
    zitten naast elkaar in het geheugen en opeenvolgende threads accessen opeenvolgende
//kleurcomponenten

__global__ void find_object_colour(const uint8_t* pixels, int* row_counts, int* col_counts) {
    int i_in_block = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x; //thread index

    if(i >= H * W * C) return;
    //zorgt ervoor dat er exact H * W * C elementen verwerkt worden en niet meer (aangezien we
        iets meer threads voorzien dan nodig:
    //laatste block bevat 1023 threads maar er worden er maar 900 gebruikt)

    __shared__ bool component_flags[1023];
    //shared mem met flags => iedere flag komt overeen met 1 thread
    //deze shared memory wordt enkel gedeeld tussen threads in eenzelfde blok

    component_flags[i_in_block] = false; //default waarde instellen

    if(i % 3 == 0) { //R component
        uint8_t r = pixels[i];
        if(r >= R_lower && r <= R_upper) {
            component_flags[i_in_block] = true; //flag is true als de kleurcomponent oke is
        }
    }else if(i % 3 == 1) { //G component
        uint8_t g = pixels[i];
        if(g >= G_lower && g <= G_upper) {
            component_flags[i_in_block] = true;
        }
    }else { //B component
        uint8_t b = pixels[i];
        if(b >= B_lower && b <= B_upper) {
            component_flags[i_in_block] = true;
        }
    }

    __syncthreads();

    if(i % 3 == 0) {
        if(component_flags[i_in_block] && component_flags[i_in_block + 1] && component_flags[
            i_in_block + 2]) {
        //we checken of alle kleurcomponenten van de pixel (RGB)
        //binnen hun range vallen (de nodige flags hiervoor zitten)
        //in een array "component_flags" in shared memory

            int row_index = (i / 3) / W;
            int column_index = (i / 3) % W;
            atomicAdd(&row_counts[row_index], 1);
            atomicAdd(&col_counts[column_index], 1);
        }
    }
}

int main(){
    ...

    int row_block_size = 1023;
    //=> 1023 ipv 1024 want 1023 is deelbaar door 3 => ik wil dat ieder block een veelvoud van 3
        threads heeft zodat
    //kleurcomponenten die tot eenzelfde pixel behoren in hetzelfde block worden berekend

```

```
    int col_block_size = 1024;
    int grid_size = ((H * W * C) / row_block_size) + 1;
    //levert 2700 blocks op (+ 1 => extra block om op te vangen dat er maar 1023 threads per block)
    //voorzien worden ipv 1024

    ...
}
```

Appendix H Implementatie CUDA V5: vroegtijdig returnen bij mismatch

```
#Volledige code: github OT_GPU_V5.cu

//er word voor iedere pixel een thread voorzien => 720 * 1280 = 921.600 threads = overklokken scenario => threads moeten in groepjes sequentieel uitvoeren
//er word nu echter vroegtijdig gestopt wanneer 1 van de kleurcomponenten niet overeenkomt => introduceert thread divergence

__global__ void find_object_colour(const uint8_t* pixels, int* row_counts, int* col_counts) {
    int i = blockIdx.x * blockDim.x + threadIdx.x; //thread index

    if(i < W * H){
        int idx = i * C; //index waarnaar thread i moet kijken in de flattened image array

        int count = 0;

        uint8_t r = pixels[idx];
        uint8_t g = pixels[idx + 1];
        uint8_t b = pixels[idx + 2];

        if(r < R_lower || r > R_upper) return;
        //iedere kleurcomponent wordt gecheckt en we returnen als 1 buiten zijn bereik valt

        if(g < G_lower || g > G_upper) return;

        if(b < B_lower || b > B_upper) return;

        count++; //hier geraken we enkel als in de vorige checks niet ge-returned is

        int row_index = idx/3 / W;
        int column_index = idx/3 % W;

        atomicAdd(&row_counts[row_index], 1);
        atomicAdd(&col_counts[column_index], 1);
    }
}
```

Appendix I Implementatie CUDA V6: asynchrone operatie

```
#Volledige code: github OT_GPU_V6.cu

//analoog aan V0 maar in deze versie wordt asynchroon telkens de volgende image ingeladen terwijl de GPU
//de rode bol zoekt
//merk op: de CPU moet wachten op de resultaten van de GPU
//=> het berekenen van het midden van de gevonden bol kan niet asynchroon

int main() {
    std::vector<std::pair<int, int>> object_coordinates;

    uint8_t* d_pixels;
    int* d_row_counts;
    int* d_col_counts;

    std::chrono::time_point<std::chrono::system_clock> StartTime = std::chrono::system_clock::now();

    uint8_t* pixels = NULL;

    for (int frameNr = 0; frameNr < 151; ++frameNr) {
```

```
    //alles loopt 1 achter dus vandaar dat we tot 150 moeten kunnen gaan
    //omwille van de NULL check is de 1ste cycle eigenlijk verloren
        std::vector<int> row_counts(H, 0);
        std::vector<int> col_counts(W, 0);

        if(pixels != NULL) {
            //stream
            cudaStream_t stream0;
            cudaStreamCreate(&stream0);

            cudaMalloc(&d_pixels, H * W * C);
            cudaMemcpyAsync(d_pixels, pixels, H * W * C, cudaMemcpyHostToDevice, stream0);

            cudaMalloc(&d_row_counts, H * sizeof(int));
            cudaMalloc(&d_col_counts, W * sizeof(int));
            cudaMemset(d_row_counts, 0, H * sizeof(int));
            cudaMemset(d_col_counts, 0, W * sizeof(int));

            // kernel gegevens
            int row_block_size = 1024;
            int col_block_size = 1024;
            int grid_size = ((H + W) / row_block_size) +1;

            find_object_colour<<<grid_size, row_block_size, 0, stream0>>>(d_pixels, d_row_counts,
                 d_col_counts);

            cudaMemcpyAsync(row_counts.data(), d_row_counts, H * sizeof(int),
                cudaMemcpyDeviceToHost, stream0);
            cudaMemcpyAsync(col_counts.data(), d_col_counts, W * sizeof(int),
                cudaMemcpyDeviceToHost, stream0);

            cudaStreamDestroy(stream0);
        }

        if(frameNr != 150) {
        // "150.ppm" bestaat niet => we mogen enkel nieuwe images inladen tot in de voorlaatste
            loop
            std::string path = "./CUDA_versions/frames/moving_ball_frames_ppm/" + std::to_string(
                frameNr) + ".ppm";
            pixels = get_image_array_ppm(path.c_str());
        }

        cudaDeviceSynchronize();
        //hier syncen: we moeten wachten tot de resultaten van de GPU klaar zijn voor we verder
            kunnen gaan
        double row_sum = 0, row_count = 0, col_sum = 0, col_count = 0;
        for (int i = 0; i < H; ++i) {
            if (row_counts[i]) { row_sum += i; row_count += 1; }
        }
        for (int j = 0; j < W; ++j) {
            if (col_counts[j]) { col_sum += j; col_count += 1; }
        }

        int row_mean = (row_count > 0) ? round(row_sum / row_count) : 0;
        int col_mean = (col_count > 0) ? round(col_sum / col_count) : 0;
        object_coordinates.push_back({row_mean, col_mean});

        //free(pixels);
        cudaFree(d_pixels);
        cudaFree(d_row_counts);
        cudaFree(d_col_counts);
    }
    free(pixels);

    std::chrono::time_point<std::chrono::system_clock> EndTime = std::chrono::system_clock::now()
        ;

    int milliseconds = std::chrono::duration_cast<std::chrono::milliseconds>(EndTime - StartTime)
        .count();

    std::cout << "Time: " << milliseconds << " milliseconds.";

    save_coordinates_to_file(object_coordinates) ;
    return 0;
}
```

References

[1] 'CUDA C++ Programming Guide'. Accessed: Feb. 24, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[2] admin, 'hoeveel FPS kan een mens zien', Humedia. Accessed: May 14, 2025. [Online]. Available: https://www.humedia.nl/waar-staat-fps-voor-en-hoeveel-fps-kan-een-mens-zien/

[3] 'P. Micikevicius, "M02: High Performance Computing with CUDA Optimizing CUDA Optimizing CUDA." Accessed: May 14, 2025. [Online]. Available: https://mc.stanford.edu/cgi-bin/images/0/0a/M02_4.pdf'